Politechnika Wrocławska
Wydział Informatyki i Telekomunikacji

Projektowanie Efektywnych Algorytmów

Sprawozdanie z zadania nr 2

Daniel Gościński 280878

Prowadzący:
dr inż. Jarosław Mierzwa

Spis treści

1. Wstęp teoretyczny 3
2. Opis algorytmów 3
3. Przykład praktyczny algorytmów 5
4. Opis implementacji 7
5. Plan eksperymentów 9
6. Warunki badawcze 10
7. Wyniki eksperymentów 10
8. Wnioski 14
9. Literatura 15

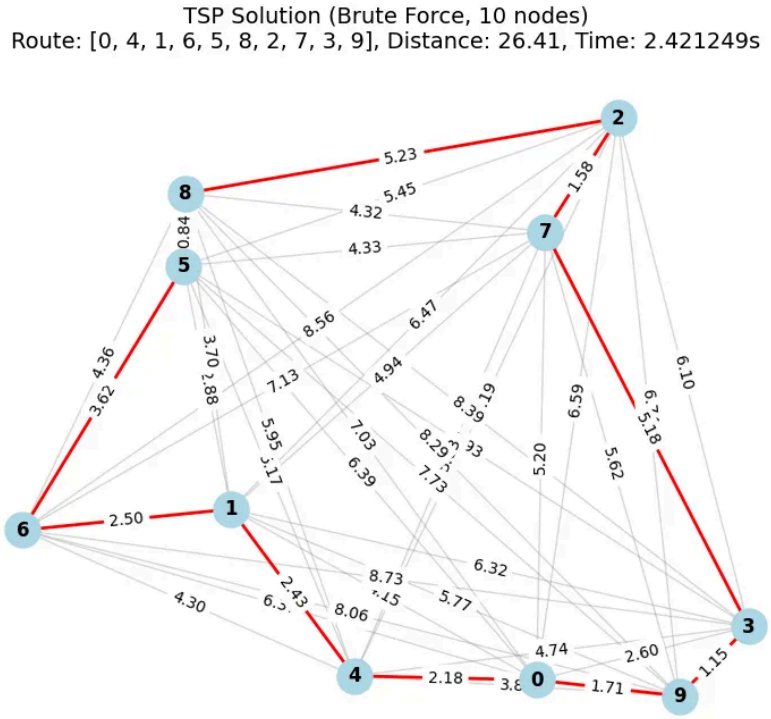
1. Wstęp teoretyczny

Problem komiwojażera to jeden z najstarszych i dobrze znanych problemów klasy NP-trudnych. Oznacza to, że nie udało się uzyskać dla niego algorytmu, który potrafiłby znaleźć optymalne rozwiązanie w czasie wielomianowym.

Zadanie te polega na znalezieniu najmniejszego cyklu Hamiltona w grafie ważonym[1]. Wyróżniamy dwa rodzaje problemu:

- Symetryczny: odległości między dwoma miastami są identyczne
- Asymetryczny: odległości między dwoma miastami nie są identyczne (Droga A-B ma inną wagę niż droga B-A)

Na potrzeby zadania rozważany jest wariant asymetryczny (**Asymmetric Travelling Salesman Problem**).



Zdjęcie 1: Przykładowa wizualizacja problemu ATSP

https://lopezyse.medium.com/graphs-solving-the-travelling-salesperson-problem-tsp-in-python-54ec2b315977

2. Opis algorytmów

Na potrzeby zadania drugiego analizowane są algorytmy wykorzystujące metodę **Podziału i Ograniczeń** (ang. **Branch-and-Bound**, BnB), która polega na przeszukiwaniu drzewa reprezentujacego przestrzeń rozwiązań [2]. Zgodnie z nazwą wykorzystywane są dwa kluczowe aspekty metody:

1. **Branching**: polega na dzieleniu rozwiązań na rozłączne podzbiory
2. **Ograniczenie**: polega na pomijaniu gałęzi drzewa o których wiadomo, że nie zawierają optymalnego rozwiązania.

Aby móc przeszukiwać przestrzeń rozwiązań należy wybrać metodę przeszukiwania. W tym zadaniu projektowym skupiono się na przeszukiwaniu wszerz (ang. **Breadth-First-Search; BreadthFS**), oraz na przeszukiwaniu najpierw najlepszy (ang. **Best-First-Search; BestFS**).

Przeszukiwanie wszerz to metoda ślepa polegająca na eksplorowaniu grafu poziom po poziomie. Zaczynając od korzenia algorytm sprawdza wszystkich swoich potomków, a następnie przechodzi kolejno do następnych poziomów drzewa. Wykorzystuje on kolejkę FIFO, zapisując w niej kolejne węzły do przeanalizowania. Ponieważ jest to algorytm ślepy, nie posiada on żadnych metod rankingu potencjału ścieżek, tylko analizuje kolejne wierzchołki aż do natknięcia się na ostatni liść. Z tego też powodu zajmuje on znaczącą ilość czasu, a kolejka w gęstym grafie może przechowywać bardzo duże ilości informacji, zajmując duże ilości pamięci podręcznej.

Przeszukiwanie najpierw-najlepszy w porównaniu do przeszukiwania wszerz wykorzystuje kolejkę priorytetową, w wyniku czego zawsze wybiera do rozwinięcia węzeł, który w danej chwili jest najbardziej obiecujący. Należy zaznaczyć, że węzeł o najtańszej ścieżce może znaleźć się na wyższym poziomie drzewa (np. po rozwinięciu jednej ścieżki koszt jej zwiększył się, w wyniku czego najmniejszy koszt ma teraz węzeł na wyższym poziomie drzewa). W wyniku działa algorytm szybko uzyskuje pierwsze legalne rozwiązanie (zgodne z zasadami ATSP), które można następnie wykorzystać jako punkt odniesienia do odcinania gorszych ścieżek. Ze względu na swoją metodę działania algorytm wykonuje się szybciej niż przeszukiwanie wszerz, co zostanie sprawdzone podczas prowadzonych badań.

W celu przyśpieszenia działania algorytmów zgodnie z zasadą **BnB** możemy wykorzystać różne metody zawężenia puli możliwych rozwiązań. Wykonuje się to poprzez wprowadzenie dolnych i górnych ograniczeń.

Górne ograniczenie (ang. Upper Bound; UB) to koszt pewnej pełnej uzyskanej ścieżki, którą następne wykorzystuje się do porównania następnych rozwiązań. Startowe UB można uzyskać poprzez np. szybkie algorytmy dające przybliżone rozwiązanie (jak np. algorytm **NN** wykonany w zadaniu 1). Następnie uzyskując następne rozwiązania możemy zastąpić istniejące UB nowym, jeśli jego wartość jest mniejsza. W wyniku takiego działania będziemy ciągle zawężać obszar poszukiwań ścieżki optymalnej.

Dolne ograniczenie (ang. Lower Bound; LB) to szacowanie najmniejszego możliwego kosztu, jaki możemy osiągnąć rozwijając daną ścieżkę. Istnieją różne metody szacowania LB, które różnią się jakością uzyskiwanego ograniczenia.

Posiadając oba ograniczenia możemy określić, czy dana ścieżka jest warta kontynuowania. Jeśli LB jest większe od UB, oznacza to, że najlepsza możliwa ścieżka

z danego węzła i tak będzie posiadać koszt większy niż aktualnie najlepsza znana droga, a w wyniku posiadania tej wiedzy możemy analizowany węzeł porzucić.

3. Przykład praktyczny algorytmów

Załóżmy macierz sąsiedztwa składającą się z 4 miast o następujących wartościach:

Tabela 1: Przykładowa macierz sąsiedztwa

	Miasto 0	**Miasto 1**	**Miasto 2**	**Miasto 3**
Miasto 0	-1	34	**78**	45
Miasto 1	21	-1	56	23
Miasto 2	67	12	-1	89
Miasto 3	32	44	18	-1

Zakładamy, że zaczynamy od miasta 0. Zaczynamy od obliczenia UB przy pomocy algorytmu najbliższego sąsiada:

0->1->3->2->0

Koszt: $34 + 23 + 18 + 67 = 142$

Nastepnie obliczamy LB korzenia. W tym przypadku (oraz w implementacji algorytmów) wykorzystane zostało obliczanie LB poprzez wyliczenie najtańszej ścieżki od węzła do korzenia, pomijając ograniczenia problemu. Wynik ten może być niemozliwy do osiągnięcia, jednak jest on wystarczający na potrzeby działania algorytmu. Dokładny opis implementacji funkcji obliczającej LB znajduje się w **sekcji 4.**

Koszt: $0 + (21 + 12 + 18) = 51$

Algorytm Best-First-Search

Do sprawdzania węzłów wykorzystywać będziemy kolejkę priorytetową

Poziom 1: rozwinięcie miasta 0

Zaczynamy od rozpisania możliwych dróg

Ścieżka	**LB**
0->1	64
0->3	78
0->2	117

Krok 1: Wybieramy 0->1 (najlepsze LB)

Ścieżka	Koszt	Wartość LB
0->1->2	90	$90 + (18) = 108$
0->1->3	57	$57 + (12) = 69$

Aktualny stan kolejki to:

Ścieżka	LB
0->1->3	69
0->3	78
0->1->2	108
0->2	117

Krok 2: Wybieramy 0->1->3

Ścieżka	Koszt	Wartość LB
0->1->3->2	75	Wszystko odwiedzone

Domykamy cykl z miasta 2 do 0

Łącznie uzyskujemy koszt $75 + 67 = 142$, który jest równy z UB (brak zmian)

Aktualny stan kolejki to:

Ścieżka	LB
0->3	78
0->1->2	108
0->2	117

Krok 3: Wybieramy 0->3

Ścieżka	Koszt	Wartość LB
0->3->1	89	101
0->3->2	63	84

Aktualny stan kolejki to:

Ścieżka	LB
0->3->2	84
0->3->1	101
0->1->2	108
0->2	117

Krok 4: Wyciągamy 0->3->2

Ścieżka	Koszt	Wartość LB
0->3->2->1	75	Wszystko odwiedzone

Domykamy cykl z miasta 1 do miasta 0

Łącznie uzyskujemy koszt $75 + 21 = 96$, który jest niższy niż aktualne UB, a więc staje się ono nowym UB

UB = 96

Aktualny stan kolejki

Ścieżka	LB
0->3->1	101
0->1->2	108
0->2	117

Krok 5: Wycinanie

Ponieważ uzyskaliśmy nowe UB analizujemy naszą kolejkę, a następnie wycinamy wszystkie drogi z LB większym od UB.

Ponieważ wszystkie elementy są większe, kolejka zostaje pusta, a to oznacza, że uzyskaliśmy wynik optymalny:

Ścieżka: 0 -> 3 -> 2 -> 1 -> 0; Koszt = 96

4. Opis implementacji

Przed przystąpieniem do wyjaśnienia implementacji należy wyjaśnić działanie dodatkowych elementów wykorzystywanych w głównych algorytmach:

1. **Struktura Node**

```
struct Node {
    std::vector<int> path;
    std::vector<bool> visited;
    int level;
    int currentCost;
    int lowerBound;
};
```

Powyższa struktura wykorzystywana jest podczas algorytmów BnB do przechowywania informacji o częściowym rozwiązaniu w drzewie rozwiązań. Przechowuje ona kolejno:

- wektor reprezentujący dotychczasową ścieżkę rozwiązania
- wektor przechowujący listę odwiedzonych miast
- informację na którym poziomie znajduje się węzeł. Na podstawie tej informacji możemy określić kiedy znajdujemy się w liściu drzewa (jeżeli poziom węzła wynosi ilość miast macierzy, znajdujemy się w liściu)
- informację o aktualnym rzeczywistym koszcie ścieżki
- informację o aktualnej wartości dolnego ograniczenia ścieżki

2. **Sposób obliczania UB**

W celu obliczania górnego ograniczenia wykorzystano utworzoną w celach zadania 1 implementację algorytmu **Najbliższego Sąsiada** (ang. Nearest Neighbor; NN). Algorytm nie daje dokładnego rozwiązania, jednak jest on wystarczający do wyznaczenia wstępnego ograniczenia obszaru rozwiązań

3. Sposób obliczenia **LB**

Utworzona została następująca funkcja do wyliczania bieżącego UB dla danego węzła:

```
int Graph::calculateBound(const Node& node, const Graph& g) {
    int bound = node.currentCost;
    for (int i = 0; i < N; ++i) {
        if (!node.visited[i]) {
            int minEdge = INT_MAX;
            for (int j = 0; j < N; ++j) {
                if (i != j && g.getEdge(i, j) != -1 && g.getEdge(i, j)
< minEdge) {
                    minEdge = g.getEdge(i, j);
                }
            }
            if (minEdge != INT_MAX) {
                bound += minEdge;
            }
        }
    }
    return bound;
}
```

Funkcja ta optymistycznie dodaje do aktualnego kosztu ścieżki sumę najtańszych przejść między nieodwiedzonymi miastami. Należy zaznaczyć, że nie zawsze wynik ten jest musi być rzeczywiście osiągalny, a ścieżka wcale nie musi być legalna (zgodna z zasadami problemu komiwojażera). Jest to jednak wystarczające, aby ocenić jakość potencjalnych ścieżek w celu wycinania nieefektywnych dróg.

3. **Główna implementacja algorytmu**

Obie implementacje (BestFS i BreadthFS) co do zasady działania są podobne, różnią się wykorzystaną strukturą do przechowywania ścieżek następnych do sprawdzenia. Dla BestFS jest to kolejka priorytetowa, zaś dla BreadthFS jest to zwykła kolejka FIFO.

Funkcja zaczyna się od obliczenia górnego ograniczenia poprzez uruchomienie na istniejącym grafie funkcję najbliższego sąsiada. Następnie przygotowywana jest struktura korzenia drzewa, która wrzucana jest do struktury przechowującej ścieżki.

Funkcja regularnie sprawdza czas w jakim się wykonuje. Jeżeli czas poszukiwania rozwiązania zajmie więcej niż 5 minut, to dalsze działania zostają przerwane, a dotychczasowe rozwiązanie zostaje zwrócone.

Główna pętla algorytmu działa tak długo aż struktura przechowywania ścieżek nie zostanie wyczerpana. W środku pętli znajdują się dwa rodzaje wykonywanych czynności.

Pierwsza z nich to obsługa sytuacji, w której sprawdzana ścieżka dotarła do liścia drzewa. W tym przypadku sprawdzany jest łączny koszt ścieżki (wliczając w to powrót do miasta startowego), który jest porównywany z UB. Jeśli koszt jest mniejszy, znaleziony został nowy najlepszy wynik, który jest zapisywany, a jego koszt staje się nowym UB.

Jeśli nie znajdujemy się w liściu, rozpoczna się proces generowania potomków węzła, w którym się znajdujemy. Proces ten polega na tworzeniu nowych zmiennych potomków, a następnie na kopiowaniu lub przypisywaniu im informacji o ich stanie (ścieżka do potomka, odwiedzone miasta, jego poziom w drzewie, koszt oraz dolne ograniczenie). Po wygenerowaniu dziecka jego LB jest porównywane z UB w celu potencjalnej natychmiastowej eliminacji ścieżek, które gwarantują gorsze wyniki.

Po wyczerpaniu kolejki FIFO/kolejki priorytetowej znaleziony wynik optymalny jest zapisywany w strukturze wynikowej, a następnie zwracany do głównej funkcji programu.

5. Plan eksperymentów

W celach badawczych utworzono program pozwalający na wykonywanie algorytmów przeszukiwania wszerz oraz najpierw-najlepszy na grafach wczytanych z pliku, lub wygenerowanych losowo. Zakłada się, że grafy zawsze są przedstawione w postaci macierzy sąsiedztw, gdzie na przekątnej macierzy pojawia się wartość -1, a w pozostałych komórkach waga ścieżki pomiędzy wybraną dwójką krawędzi. Z wykorzystaniem programu wykonano badanie pomiaru czasu wykonywania obu algorytmów dla tego samego grafu.

Badanie wykonano poprzez zmierzenie czasu dla 6 różnych rozmiarów instancji w przedziale <10;16>. Dla każdego rozmiaru pomiary wykonano dla 100 losowych grafów, a wyniki uśredniono. Ustanowiono maksymalny dozwolony czas działania algorytmu na 5 minut, po których algorytm zostawał przerwany, a jego wyniki nie były brane pod uwagę w wyliczaniu średniego czasu znajdywania rozwiązania. W takich przypadkach zliczano ilość przerwanych algorytmów na wskutek upływu dozwolonego czasu.

Należy zaznaczyć, że czas mierzony był od momentu rozpoczęcia się głównej pętli funkcji. Oznacza to, że czas wyznaczenia UB metodą NN, jak i czas potrzebny na utworzenie korzenia drzewa nie były brane pod uwagę.

6. Warunki badawcze

Badania zostały przeprowadzone na następujących podzespołach, następującej wersji systemu operacyjnego oraz wersji wykorzystanego kompilatora

Tabela 2: Warunki badawcze

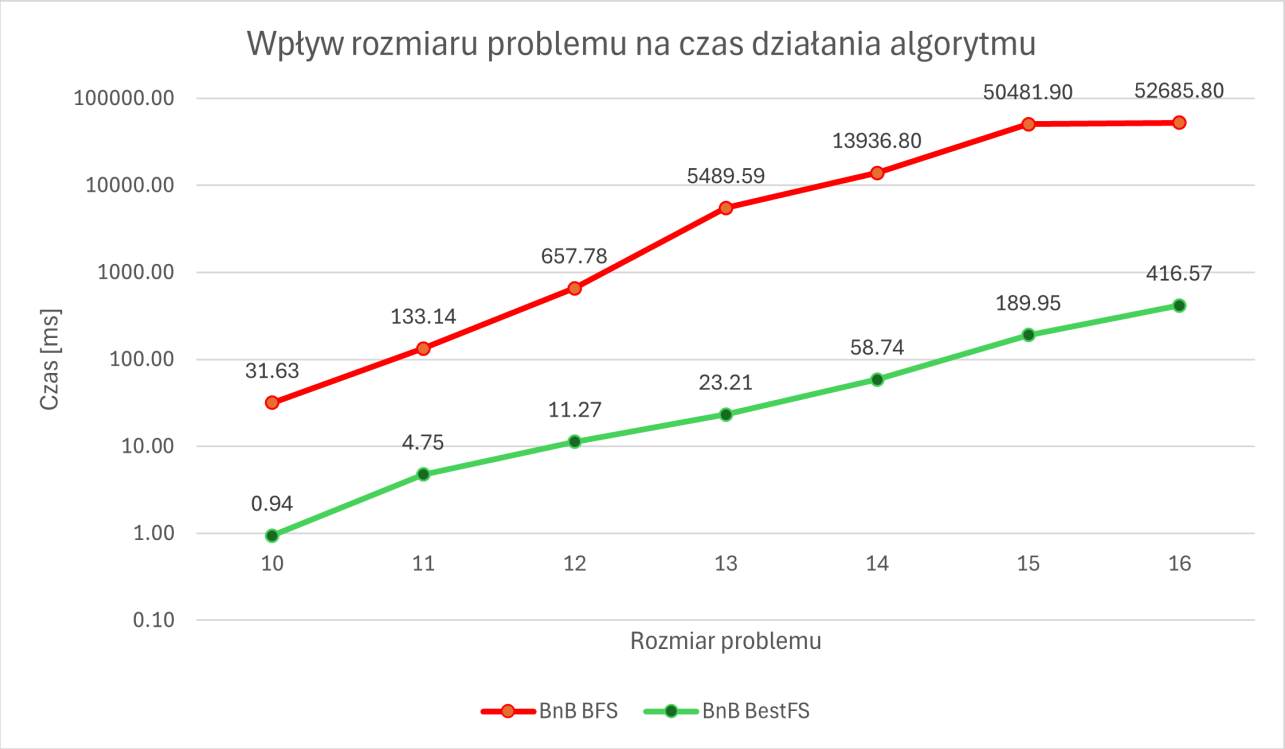
Procesor	AMD Ryzen 5 3600
Karta graficzna	NVIDIA GeForce RTX 5070
RAM	Kingston FURY 16GB (2x8GB) 3600MHz CL16 Crucial 16GB (2x8GB) 3600MHz CL16
Płyta główna	MSI B450-A PRO MAX
Windows	Windows 11 Version 25h2 (OS Build 26200.8246)
Kompilator	gcc version 15.2.0 (Rev9, Built by MSYS2 project)

7. Wyniki eksperymentów

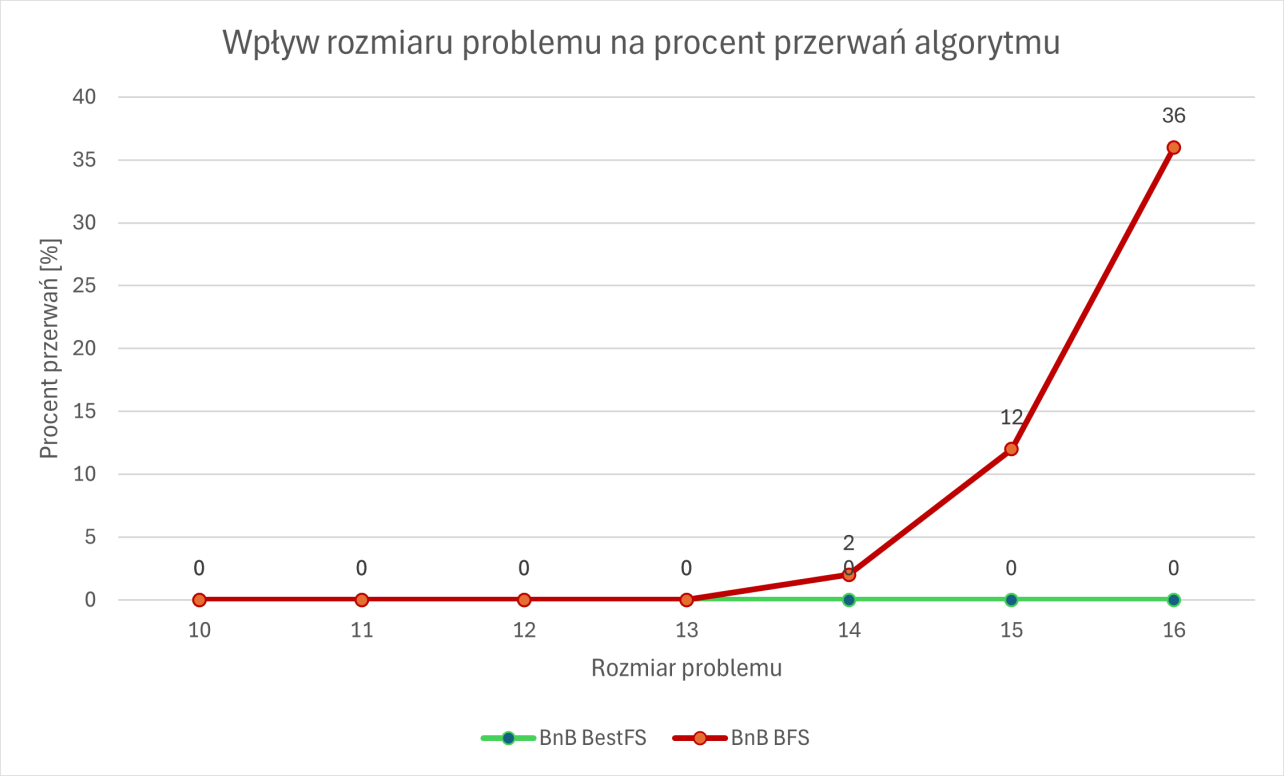
Ponieważ skrót obu algorytmów to **BFS**, w wynikach rozróżnia się je jako **BestFS** dla przeszukiwania najpierw-najlepszy, oraz **BFS** dla przeszukiwania wszerz.

Tabela 3: Czas wykonywania algorytmów przeszukiwania oraz procent przerwań w zależności od rozmiaru problemu N

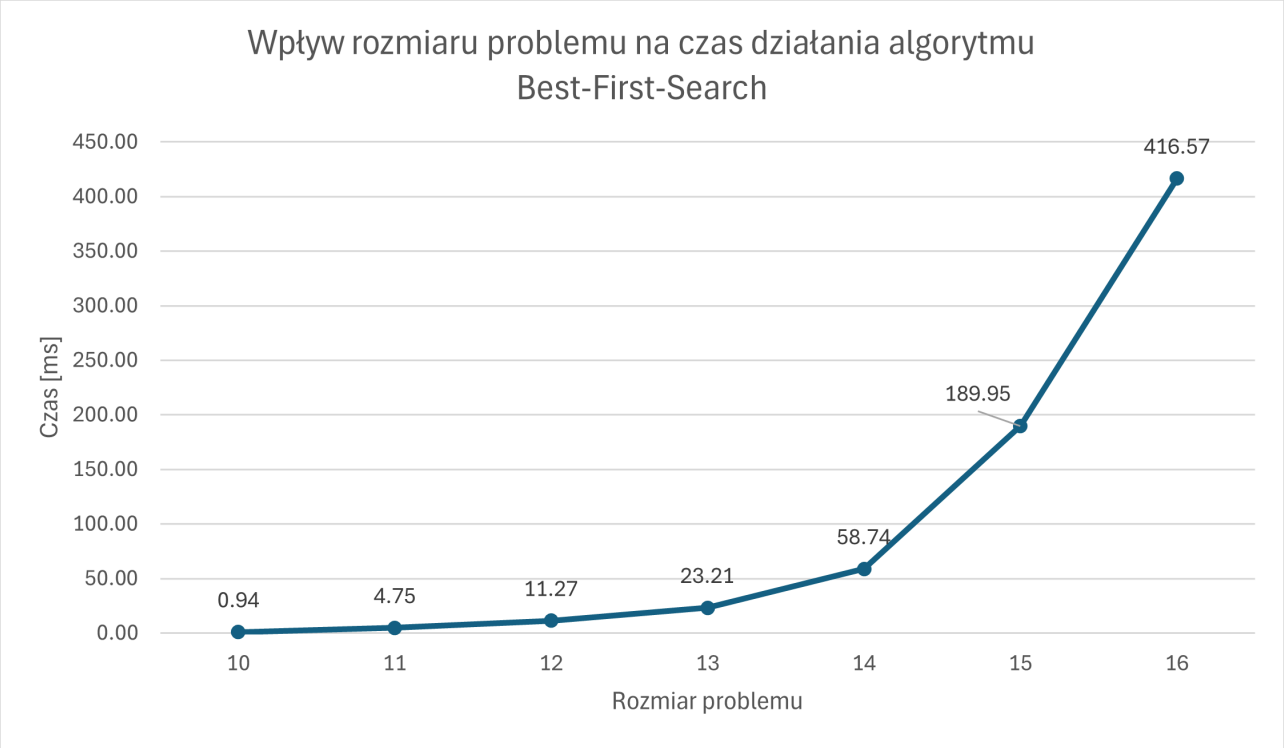
Algorytm	N	Czas [Milisekundy]	Przerwane [%]
BnB BestFS	10	0.94	0
BnB BestFS	11	4.75	0
BnB BestFS	12	11.27	0
BnB BestFS	13	23.21	0
BnB BestFS	14	58.74	0
BnB BestFS	15	189.95	0
BnB BestFS	16	416.57	0
BnB BFS	10	31.63	0
BnB BFS	11	133.14	0
BnB BFS	12	657.78	0
BnB BFS	13	5489.59	0
BnB BFS	14	13936.80	2
BnB BFS	15	50481.90	12
BnB BFS	16	52685.80	36



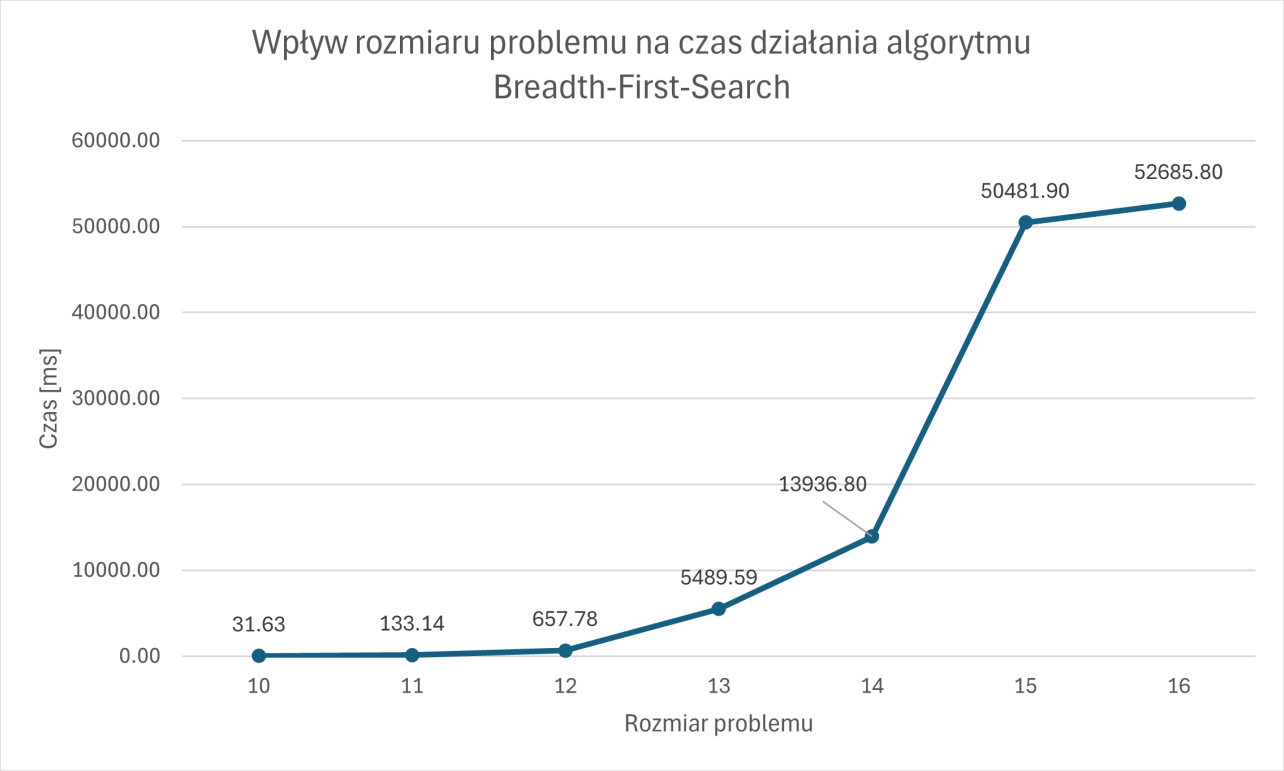
Wykres 2: Średni czas wykonywania algorytmu BFS oraz BestFS w zależności od rozmiaru problemu N



Wykres 3: Wpływ rozmiaru problemu N na procent przerwań algorytmów



Wykres 4: Średni czas wykonywania algorytmu BestFS w zależności od rozmiaru N



Wykres 5: Średni czas wykonywania algorytmu BFS w zależności od rozmiaru N

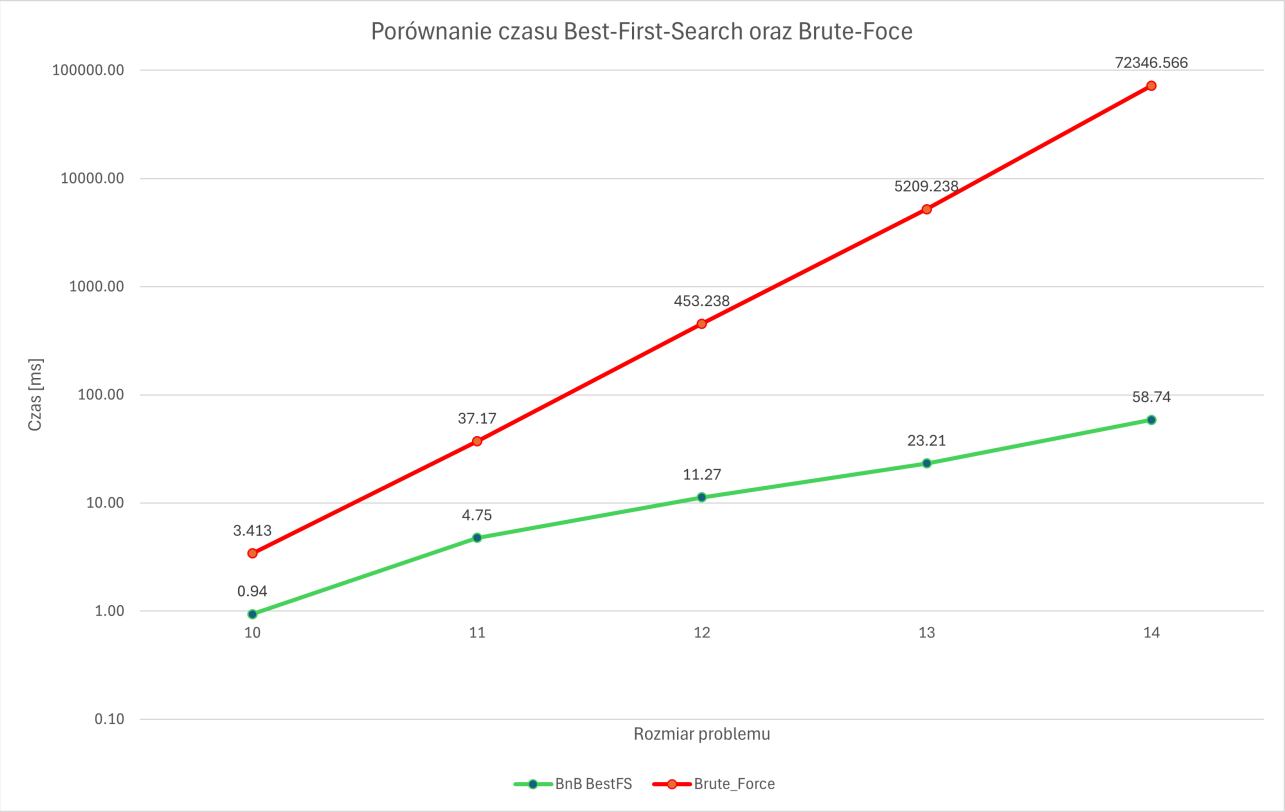
Jak możemy zobaczyć na wykresie nr. 2 obie linie w przybliżeniu są liniami prostymi, co oznacza złożoność wykładniczą. Można również zaobserwować przepaść pomiędzy algorytmami, gdzie wyniki dla danego problemu różnią się o prawie dwa rzędy wielkości. Widać tutaj przewagę zastosowania kolejki priorytetowej, która w lepszy sposób pozbywa się ogromnej części drzewa przeszukiwań.

Wykres nr. 3 również pokazuje znaczącą różnicę w działaniu obu algorytmów. Zaczynając od rozmiaru problemu na poziomie 14 miast przeszukiwanie wszerz zaczyna nie zdążać wykonać swoich operacji w zadanym czasie 5 minut. Pokazuje to nieprzewidywalność algorytmu, który dla większych rozmiarów jest bardzo zależny od układu danych. Problemu tego nie zaobserwowano dla tych samych danych w przypadku wyszukiwania najpierw-najlepszy.

Na wykresie 2 oraz 5 można zaobserwować pewną anomalię, gdzie przyrost między rozmiarami 15 i 16 nie jest podobny do wcześniejszych przyrostów. Należy zauważyć, ze jest przy tych samych rozmiarach na wykresie 3 widzimy znacząco rosnącą ilość przerwań w działaniu algorytmu BFS. Łącząc oba fakty można zinterpretować widniejącą zmianę jako błąd przeżywalności. Ponieważ generowane grafy są losowe, tylko te „łatwiejsze” problemy zostały rozwiązane w czasie, a aż 1/3 problemów potrzebowałaby ponad 5 minut.

Uzyskane wyniki algorytmu BestFS możemy również porównać z czasami przeglądu zupełnego uzyskanego w zadaniu 1. Możemy tutaj zobaczyć, jak zmiana

podejścia do przeglądu obszaru rozwiązań może znacząco wpłynąć na czas znajdywania optimum.



Wykres 6: Porównanie czasów wykonania algorytmu BestFirstSearch i BruteForce

8. Wnioski

Wykonane badania projektowe pozwoliły na zrozumienie działania metody Podziału i Ograniczeń, oraz na jej wpływ na szukanie rozwiązania Asymetrycznego Problemu Komiwojażera.

Utworzone dwie metody posiadają swoje zalety oraz wady. Przeszukiwanie wszerz jest prostszą implementacją wykorzystującą prostą kolejke FIFO, bez potrzeby specjalnego porównywania rezultatów między węzłami. Przeszukuje ono systematycznie drzewo, nie pomijając jego części, a jego wynik jest mniej zależny od implementacji uzyskiwania dolnego ograniczenia. Jego znaczącą wadą jest słabe odcinanie ścieżek, ponieważ zanim pozyskane będzie lepsze górne ograniczenie, algorytm musi przebyć dużą część drzewa. W wyniku tego algorytm zużywa bardzo dużo pamięci RAM, oraz działa bardzo długo, a w pesymistycznych sytuacjach może on dojść nawet do przeglądu zupełnego.

Metoda przeszukiwania najpierw-najlepszy jest mądrzejszy od wcześniejszego przeszukiwania ślepego, w wyniku czego algorytm zawsze stara się iść w kierunku dobrego rozwiązania. Wykonuje on odcinanie o wiele lepiej, w wyniku czego oszczędza on znacznie na czasie wykonywania, oraz na wykorzystywanej pamięci (odcięte węzły zostają usunięte z pamięci). Jego największą wadą jest zależność od

heurystyki LB, która znacząco wpływa na odcinanie węzłów. W pesymistycznych sytuacjach algorytm nadal jest w stanie zużywać duże ilości pamięci, a w najgorszych przypadkach również może dojść do przeglądu zupełnego.

Jak możemy również zauważyć, dla wyznaczonego maksymalnego czasu 5 minut algorytm Breadth-First-Search zaczyna mieć problemy już dla rozmiaru 14. W zależności od rozkładu danych algorytm jest w stanie nie zdążyć wykonać wszystkich potrzebnych operacji. Dla drugiego algorytmu starano się sprawdzić empirycznie dla jakiego rozmiaru doszło by do podobnej sytuacji, jednak nie udało się znaleźć rozmiaru który zakończyłby przedwcześnie działanie z powodu niewystarczającego czasu (działanie kończyło się na wskutek braku wystarczającej ilości pamięci podręcznej).

Można również zastanowić się jak istnienie rozwiązania początkowego wpływałby na czas wykonania algorytmów. Rozwiązanie początkowe w naszym przypadku jest rozwiązaniem poznanym przy pomocy algorytmu NN, które stosujemy jako górne ograniczenie. Możemy zamienić to rozwiązanie np. na maksymalną wartość zmiennej typu **int**. Logicznym będzie, że w takim przypadku musimy spośród obszaru rozwiązań pozyskać pierwsze rozwiązanie, aby móc rozpocząć wykonywać jakiekolwiek obcinanie węzłów. W przypadku przeszukiwania wszerz może okazać się, że ta zmiana spowoduje na tyle dużym zapotrzebowaniem na pamięć przez algorytm, że rozmiary grafów, które wcześniej byliśmy w stanie rozwiązań staną się nierozwiązywalne.

To samo możemy powiedzieć o metodzie najpierw-najlepszy. W kolejce będą przechowywane wszystkie ścieżki tak długo, aż nie uzyskamy pierwszego rozwiązania, które pozwoli na wykonywanie jakiegokolwiek porównywania, oraz obcinania.

Wprowadzenie w obu przypadkach dowolnego poznanego rozwiązania pozwala na przyśpieszenie działania algorytmu. Skala przyśpieszenia zależna jest od tego, jak blisko rozwiązania optymalnego jest wprowadzone rozwiązanie (co nie zawsze jest w naszej kontroli).

9. Literatura

[1] S. Lin i B. W. Kernighan, „An Effective Heuristic Algorithm for the Traveling--Salesman Problem”, *Operations Research*, t. 21, nr 2, s. 498–516, 1973, Dostęp: 25 marca 2026. [Online]. Dostępne na: http://www.jstor.org/stable/169020

[2] dr inż. Tomasz Kapłon, „Wykład Projektowanie Efektywnych Algorytmów”.